# Retrieval-Assisted Instantiation of Natural-Language Optimization Problems

Soroush Vahidi[1*]

[1*]Department of Computer Science, Ying Wu College of Computing, New Jersey Institute of Technology, University Heights, Newark, 07102-1982, NJ, USA.

Corresponding author(s). E-mail(s): sv96@njit.edu (ORCID: 0000-0003-1934-6282);

**Abstract**

**Purpose:** Many optimization problems are first described in natural language before being formalized as mathematical models. This paper studies retrieval-assisted optimization-problem instantiation: retrieving a compatible schema from a fixed catalog and deterministically grounding its scalar parameters from text, to separate schema-identification difficulty from downstream scalar-grounding difficulty. **Methods:** We evaluate classical lexical retrieval (TF–IDF, BM25, LSA) and a pretrained dense retriever (BGE-M3) for schema identification on the NLP4LP benchmark (331 queries, a 335-candidate schema catalog), paired with a deterministic, inference-time-LLM-free numeric-extraction and typed-greedy slot-assignment procedure. An oracle schema-selection control isolates downstream grounding difficulty from retrieval error, with paired bootstrap and McNemar tests for robustness. **Results:** Schema retrieval is already strong (Schema R@1 = **0.9094** for TF–IDF, **0.9456** for BGE-M3). BGE-M3 paired with deterministic grounding reaches the highest InstantiationReady among non-oracle configurations (**0.8248**), while an oracle control with perfect retrieval reaches only **0.8489**, a modest additional gain. An exact residual-error decomposition further shows that even when the correct schema, full slot coverage, and correct coarse types are all achieved, **64.7%** of queries still contain at least one scalar value outside the accepted tolerance, identifying fine-grained value-to-slot correctness – not schema retrieval or coarse type identification – as the dominant remaining bottleneck. **Conclusion:** Fixed-catalog schema retrieval is not the limiting factor in this benchmark; deterministic scalar grounding, and specifically precise value-to-slot correctness, remains the primary open challenge.

Restricted structural and solver-backed subset studies suggest downstream usefulness on compatible cases without establishing benchmark-wide solver readiness or open-domain generalization.

**Keywords:** natural language processing, optimization modeling, knowledge representation, information retrieval, semantic grounding, intelligent information systems

**Acknowledgements.** The author is deeply grateful to his mother for her continuous emotional support and thanks his Ph.D. advisor, Professor Ioannis Koutis, for his support, guidance, and encouragement. The author gratefully acknowledges Anders Borum for providing complimentary lifetime access to Secure ShellFish, which was used in preparing this manuscript.

# 1 Introduction

Many real-world engineering and operations problems are first described in natural language before they are formalized as optimization models. Analysts specify resource limits, costs, capacities, proportions, and operational rules in text long before these elements become a mathematical program. Bridging that gap is a central knowledge-engineering concern: unstructured textual knowledge must be aligned with an appropriate formal structure [1, 24, 25].

This paper studies a narrower intermediate task: *retrieval-assisted optimization-problem instantiation* over a fixed catalog of structured schemas. Rather than synthesizing an arbitrary program, the system retrieves the most compatible schema and then deterministically grounds that schema's scalar parameters from numeric evidence in the text. Eligible slots are determined by the predicted schema, so downstream evaluation remains retrieval-dependent by construction and the two stages can be measured separately [2, 3, 28]. Lexical retrieval systems can select plausible templates but do not by themselves recover the quantities needed to instantiate them. End-to-end LLM systems can generate formulations or solver code [3, 4, 37], yet they conflate several failure modes, obscuring which intermediate capabilities are reliable [5, 13, 20]. The resulting research question is therefore: in a fixed-catalog setting, does residual difficulty lie mainly in schema retrieval or in scalar grounding once a schema is identified?

We address this with a transparent two-stage pipeline. Classical lexical or pretrained dense retrieval ranks the catalog and selects a top-1 schema; rule-based extraction, coarse typing, and deterministic no-reuse assignment then fill eligible scalar slots. Intermediate decisions are inspectable for oversight workflows [7, 23]. The intended practical path is: natural-language description → retrieve a schema → ground numeric evidence → inspect → human/modeling verification → optional solver or model building. This paper evaluates the retrieval and scalar-grounding portion under a fixed catalog.

On NLP4LP [2, 3], TF–IDF already achieves Schema R@1 = 0.9094 and BGE-M3 reaches 0.9456. BGE-M3 with deterministic grounding attains *InstantiationReady* = 0.8248, while an oracle control reaches only 0.8489, indicating that scalar grounding—not schema identification—is the larger residual challenge. Restricted structural and solver-backed subset studies suggest usefulness on compatible cases without claiming benchmark-wide solver readiness.

Novelty does not lie in TF–IDF, BM25, LSA, BGE-M3, or ordinary rule extraction. The contribution is a retrieval-conditioned formulation with deterministic, inference-time LLM-free scalar grounding, controlled lexical/dense/oracle separation of retrieval from grounding, and an exact residual-error decomposition with a StrictInstantiationReady diagnostic. Empirically, fixed-catalog retrieval is strong on NLP4LP, while fine-grained value-to-slot correctness dominates residual failure. Concretely, this paper contributes: (1) a *retrieval-conditioned schema-instantiation formulation* in which the retrieved schema determines eligible scalar slots; (2) an *inference-time LLM-free deterministic grounding procedure* with inspectable extraction, typing, and no-reuse assignment; (3) a *diagnostic evaluation* under lexical, dense, and oracle schema selection that identifies scalar grounding as the larger residual challenge; and (4) *reproducibility and downstream validation* spanning robustness, structural, solver-backed, and external component-level evidence with explicitly scoped claims.

Section 2 reviews related work. Section 3 presents the formulation and pipeline. Section 4 reports experiments. Section 5 concludes.

## 2 Related Work

Recent advances in natural-language-to-optimization modeling reflect a shift from entity extraction to end-to-end generative systems, and increasingly to structured grounding and retrieval. We organize this literature around four primary axes that situate the present study.

### 2.1 End-to-End LLM Optimization-Model Generation

An increasingly prominent direction uses large language models (LLMs) to produce complete optimization formulations, solver code, or executable models from natural-language descriptions. Early benchmark efforts such as NL4Opt focused on entity extraction and formulation generation [25], while systems like OptiMUS [3] and OptLLM [37] demonstrated that LLMs could generate and iteratively debug linear and mixed-integer programming models. Recent work has increasingly focused on fine-tuning and synthetic data generation. Systems such as LLMOPT [14] fine-tune models on structured optimization formulations, and ORLM develops a customizable training framework with solver-guided feedback [13]. To enrich training corpora, Opt-MATH introduces a scalable bidirectional data synthesis framework [20]. For more advanced training, Solver-Informed Reinforcement Learning (SIRL) employs verifiable solver feedback to train LLMs via reinforcement learning [9], and OR-R1 automates modeling via test-time reinforcement learning [11]. The DeepOR architecture further proposes a deep-reasoning foundation model specifically for optimization modeling

[31]. Still more recent agentic and experience-library systems extend this line: OptiMAI coordinates formulator, planner, coder, and critic agents with UCB-based debug scheduling and reports strong end-to-end accuracy on NLP4LP [29]; AlphaOPT accumulates solver-verified modeling insights in a self-improving experience library without parameter updates [16]; and ORThought couples expert-guided chain-of-thought modeling with a logistics-focused benchmark and diagnostic evaluation [33]. These systems target complete formulation and/or solver-code generation with end-to-end objective or execution metrics, rather than fixed-catalog schema-conditioned scalar instantiation.

## 2.2 Retrieval, Memory, and Solver-Feedback Systems

To improve the correctness of generated models, recent systems increasingly incorporate retrieval, memory, and solver feedback into the generative process. OPT2CODE combines retrieval and code generation to translate linear programs into solver-executable formulations [4], while Autoformulation maps the problem to a search over hierarchically decomposed components using Monte-Carlo tree search [5]. Extending test-time search, Yu et al. explicitly decompose the generation process into instruction, assumptions, formulation, and solver-code stages using an uncertainty-aware search mechanism [34]. Under multi-agent designs, Chain-of-Experts coordinates role-specialized agents [30], and Zhang et al. propose a dual-cluster memory agent to separate modeling and coding knowledge for structured retrieval of historical solutions [38]. Furthermore, experience replay mechanisms have been introduced to maintain persistent correction memory from solver tracebacks [32]. These systems use retrieval primarily to augment generative LLM pipelines; they generally target open-ended formulation, extending to broader tasks such as solver-independent automated problem formulation [18] and question partitioning [21].

## 2.3 Binding and Structured Grounding

As benchmarks mature, researchers have exposed a steep performance drop when transitioning from simple puzzles to complex formulations [19]. This drop has motivated a critical distinction between a system's *modeling* ability (selecting mathematical structure) and its *binding* ability (grounding specific numeric parameters and data from text into that structure). Recent work by Gao et al. explicitly identifies data binding as a major limitation of generative text-to-optimization systems, reporting an effective binding limit in open-ended formulations [12]. Earlier component-level approaches such as Ner4Opt [15] framed optimization modeling as an information-extraction problem over natural-language entities and quantities. In the broader data and knowledge engineering (DKE) literature, structured grounding and schema matching remain central challenges [1, 28, 35]. These principles are increasingly relevant for optimization tasks, such as automated equivalence checking [36], and parallel broader trends in semantic query answering and conceptual design [6, 26].

Our study addresses a narrower but complementary setting to the binding challenges identified by Gao et al. [12]. We do not claim priority for the observation that binding is difficult. Rather than generating an open-ended formulation and then

**Table 1** Task-level comparison landscape for recent natural-language optimization modeling systems. This table is deliberately *not* a leaderboard: systems differ in output type, supervision, and evaluation protocol, so accuracy numbers are omitted. Abbreviations: Ret. = retrieval/memory; Sol. = solver feedback during generation or training; FT = fine-tuning or RL training; Open = public code and/or checkpoint available at the time of writing; Comp. = directly comparable to the InstantiationReady / Exact20 (on hits) protocol of this paper.

| Method | Year | Task | Output | Ret. | Sol. | FT | Open | Comp. |
|---|---|---|---|---|---|---|---|---|
| OptiMUS | 2024 | End-to-end (MI)LP | Model+code | no | yes | no | yes | no |
| OptLLM | 2024 | NL→model/code | Model+code | no | yes | no* | yes | no |
| Chain-of-Experts | 2024 | Multi-agent OR | Model+code | no | yes | no | yes | no |
| LLMOPT | 2025 | Fine-tuned modeling | Model+code | no | yes | yes | yes | no |
| OPT2CODE | 2025 | Retrieval+codegen | Solver code | yes | yes | no | yes | no |
| ORLM | 2025 | Training+feedback | Model+code | no | yes | yes | yes | no |
| OptMATH | 2025 | Data synthesis | Model+code | no | yes | yes | yes | no |
| SIRL | 2025 | Solver-informed RL | Model+code | no | yes | yes | yes | no |
| OR-R1 | 2026 | Test-time RL | Model+code | no | yes | yes | part.† | no |
| DeepOR | 2026 | Foundation OR | Formulation | — | — | yes | no | no |
| OptimAI | 2025 | Multi-agent NL-OR | Model+code | no | yes | no | — | no |
| AlphaOPT | 2025 | Experience library | Model+code | yes | yes | no‡ | yes | no |
| ORThought | 2026 | Logistics CoT | Model+code | no | yes | no | yes | no |
| Ours | 2026 | Fixed-catalog inst. | Schema+scalars | yes | no | no | yes | yes |

*OptLLM supports both prompting and fine-tuned variants. †part. = partial: official code public; no trained checkpoint located for evaluation in this work. ‡Library learning without weight updates. "—" denotes information not established from a verified public artifact at the time of writing. Direct comparability (Comp.) is "yes" only for systems that share our fixed-catalog scalar-instantiation task and InstantiationReady / Exact20 protocol; end-to-end solver accuracy is a different metric.

binding its data, we retrieve one schema from a fixed catalog and measure how retrieval errors alter the scalar slot inventory available to deterministic grounding—via oracle schema selection, StrictInstantiationReady, and an exact residual-error decomposition.

## 2.4 Position of the Present Work

Relative to the families above, this paper isolates fixed-catalog schema retrieval from schema-conditioned scalar grounding under InstantiationReady / Exact20. Grounding is deterministic and inference-time LLM-free; retrieval is evaluated with both lexical and BGE-M3 backbones. This is a cost–capability tradeoff relative to open-ended generative modeling, not a claim of universal superiority. Table 1 summarizes the landscape without mixing incomparable accuracy numbers. Because no external system exposes a directly comparable InstantiationReady protocol, Section 4.4 reports a matched same-task comparison among already-implemented grounding alternatives under fixed retrieval.

# 3 Methodology

## 3.1 Problem Formulation

We study retrieval-assisted instantiation for natural-language optimization descriptions. Given a query, the system returns (i) a top-1 schema from a fixed catalog and (ii) scalar parameter assignments for that schema—an intermediate step before full model synthesis [2, 3, 25]. Schema retrieval ranks $\mathcal{S} = \{s_1, \ldots, s_M\}$ and returns

$$\hat{s} = \arg\max_{s \in \mathcal{S}} \text{score}(q, s). \tag{1}$$

Parameter instantiation then extracts numeric mentions and assigns them to scalar slots of $\hat{s}$ with deterministic rules. Incorrect retrieval can mis-specify the eligible slot set even when the query contains usable numeric evidence.

A *schema* here is the retrieved template together with its scalar parameter structure. The benchmarked setting does not synthesize a complete program, recover all variables/constraints, or reconstruct list-/vector-/dictionary-valued objects. Scope is restricted in three deliberate ways: (1) scalar numeric parameters only; (2) deterministic grounding (no generative LLM or learned extractor; no fine-tuning), with retrieval evaluated via lexical methods and pretrained BGE-M3; (3) schema-conditioned scalar recovery rather than end-to-end generation. Structural and solver-backed subset studies later assess practical relevance without redefining this core task (Section 4.8).

Let $K_G(q)$ be the scalar gold keys for query $q$ and $P(\hat{s})$ the scalar keys of $\hat{s}$. Evaluation uses the eligible set

$$E(q, \hat{s}) = K_G(q) \cap P(\hat{s}), \tag{2}$$

with gold map $y_q : K_G(q) \to \mathbb{R}$ and predictions $\hat{y}_q$ on assigned slots $A(q) \subseteq E(q, \hat{s})$. Retrieval quality is top-1 schema match; downstream quality is how well eligible scalars can be filled from text. As shown later, retrieval on NLP4LP is already strong, while residual difficulty concentrates in scalar grounding.

## 3.2 Retrieval-Based Schema Identification

The first stage selects the most compatible candidate from a fixed catalog of benchmark-induced schema documents. Each candidate is indexed by a schema identifier and paired with its textual description. Given a query, the retriever scores every candidate and returns a ranked list; only the top-1 schema is used for grounding. The catalog contains 335 documents: the task is fixed-catalog identification of the structural template for downstream grounding, not open-domain search and not coarse family classification into a handful of labels. Through its identifier, the selected schema exposes the eligible scalar parameter inventory [2, 3, 25]. Ranking uses natural-language schema documents rather than abstract class names, so retrieval quality directly shapes the structural conditions of the second stage.

We evaluate BM25 [27], TF–IDF cosine retrieval [22], LSA [10], and pretrained dense BGE-M3 [8] (Table 2). Classical methods are CPU-compatible and untrained

**Table 2** Retrieval methods used for schema identification.

| Method | Representation | Role in pipeline |
|---|---|---|
| BM25 | Probabilistic lexical weighting | Sparse lexical baseline |
| TF–IDF | Weighted bag-of-words cosine similarity | Primary lexical baseline |
| LSA | Latent projection of TF–IDF features | Lower-dimensional lexical-semantic baseline |
| BGE-M3 | Pretrained dense text embeddings | Modern semantic retrieval baseline |

on this benchmark; BGE-M3 is used off-the-shelf. For query $q$,

$$\hat{s}(q) = \arg\max_{s \in \mathcal{S}} \text{score}(q, s). \tag{3}$$

Only $\hat{s}(q)$ is passed downstream. An *oracle* control substitutes the gold schema to expose grounding under perfect retrieval. A *random* reference is $1/335 \approx 0.0030$ for retrieval-only evaluation, or a uniformly chosen schema passed through the same grounding pipeline for downstream evaluation. Wrong retrieval can shrink or misalign the eligible slot set; strong retrieval supplies a more appropriate inventory for deterministic grounding.

## 3.3 Deterministic Parameter Instantiation

Given a query and predicted schema, the second stage produces slot–value assignments for eligible scalar parameters. It does not generate a complete model or recover variables, constraints, or structured objects. A fixed rule-based extractor recognizes integers, decimals, percentages, currency-like forms, and comma-separated magnitudes; masked tokens in degraded settings are preserved as numerically unknown. Mentions receive a coarse type from {*percent*, *integer*, *currency*, *float*} via surface cues. Eligible slots come from the predicted schema's scalar keys; expected types are inferred from parameter names with the same four-way taxonomy.

Typed greedy matching is the production assignment rule. With eligible slots $\mathcal{P} = \{p_1, \ldots, p_m\}$ and mentions $\mathcal{T} = \{t_1, \ldots, t_n\}$,

$$t_i^* = \arg\max_{t \in \mathcal{T} \setminus \mathcal{U}_{i-1}} \text{compat}(p_i, t), \tag{4}$$

where $\mathcal{U}_{i-1}$ are previously used mentions. Compatibility prioritizes type agreement with deterministic tie-breaking; assigned mentions are not reused. Section 4.4 compares this rule to other already-implemented same-task alternatives under fixed TF–IDF retrieval (typed greedy and constrained assignment share the final ratio-aware extractor; others retain intrinsic enriched extractors). Typed greedy remains strongest on InstantiationReady / StrictInstantiationReady; some alternatives raise conditional Exact20 by abstaining more. A selective schema-and-grounding reranking

**Algorithm 1** Deterministic parameter instantiation: extract and type mentions; obtain eligible scalar slots from the predicted schema; assign without token reuse under a deterministic compatibility rule.

**Require:** Query text $q$, predicted schema $\hat{s}$
**Ensure:** Slot–value mapping $A$
Extract numeric mentions $\mathcal{T}$ from $q$
Infer a coarse type for each mention in $\mathcal{T}$
Obtain parameter keys from schema $\hat{s}$
Restrict to eligible scalar slots $\mathcal{P} = \{p_1, \ldots, p_m\}$
Infer an expected type for each slot in $\mathcal{P}$
Initialize assignments $A \leftarrow \emptyset$ and used-mention set $\mathcal{U} \leftarrow \emptyset$
**for** each slot $p_i \in \mathcal{P}$ **do**
    Define candidate set $\mathcal{C}_i \leftarrow \mathcal{T} \setminus \mathcal{U}$
    **if** $\mathcal{C}_i = \emptyset$ **then**
        **continue**
    **end if**
    Score mentions in $\mathcal{C}_i$ using a deterministic slot–mention compatibility rule
    Select the highest-ranked mention $t_i^*$
    Assign $A[p_i] \leftarrow \text{value}(t_i^*)$
    Update $\mathcal{U} \leftarrow \mathcal{U} \cup \{t_i^*\}$
**end for**
**return** $A$

diagnostic (not a grounding-only comparison) showed an apparent Instantiation-Ready gain driven by wrong-schema artifact (Section 4.6). Algorithm 1 and Figure 1 summarize the common backbone.

Optional lexicon-based role cues for tie-breaking exist but are not part of the primary main-benchmark configurations and carry no formal correctness guarantee. A numeric-extraction ablation corrects multiplicative ratio words (e.g., “twice”, “triple”) so extracted values are scaled deterministically; experiments show gains in TypeMatch and InstantiationReady with retrieval unchanged.

## 3.4 Evaluation-Oriented Design Choices

Eligible slots are taken from the *predicted* schema, not the gold schema, so retrieval errors propagate into coverage and readiness without using gold information to choose which schema is instantiated. Scalar-only evaluation provides a common substrate across templates (Section 3.1). Oracle and random references retain the interpretive roles in Section 3.2; the oracle is a control rather than a formal upper bound, because InstantiationReady need not be monotonic in schema correctness (Section 4.8 reports a non-monotonic case on the 20-instance solver subset). Core metrics are solver-free and grounding is deterministic; executability is examined separately on restricted subsets. InstantiationReady is a diagnostic readiness criterion, not a guarantee of model correctness or solver compatibility.

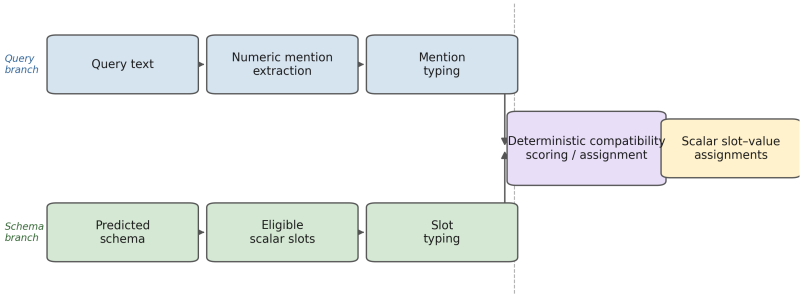


**Fig. 1** Schema-conditioned deterministic parameter instantiation. Numeric mentions are extracted from the query, typed using surface cues, filtered through the scalar slot inventory of the predicted schema, and assigned by a no-reuse compatibility rule. Alternative deterministic compatibility rules can be layered on this backbone.

# 4 Experiments

## 4.1 Experimental Setup

We evaluate on NLP4LP [2, 3], a curated gated public benchmark of LP/MILP descriptions pairing each instance with a gold parameter record and reference solver code. Unless noted, results use the 331-query test split; each query has one gold schema and is ranked against the 335-document catalog (random top-1 chance $1/335 \approx 0.0030$). Three query variants probe robustness: *orig* (full text), *noisy* (controlled lexical corruption and numeric masking), and *short* (first sentence only).

Retrieval uses BM25, TF–IDF, LSA, and off-the-shelf BGE-M3 (no benchmark fine-tuning); only top-1 schemas enter grounding, with an oracle control for perfect selection. Downstream grounding follows Section 3.3, restricted to scalar slots induced by the predicted schema. Reported configurations are: (i) TF–IDF typed-greedy with baseline extraction; (ii) TF–IDF typed-greedy with ratio-aware extraction; (iii) BGE-M3 with the same ratio-aware grounding; (iv) oracle typed-greedy. Comparing (ii) and (iii) holds grounding fixed and varies only retrieval.

*Schema R@1* is the fraction of queries with correct top-1 schema. For downstream metrics, let $G_i$ be eligible scalar gold slots after intersecting gold scalars with predicted-schema slots, and $A_i \subseteq G_i$ the assigned subset. Then

$$\mathit{Coverage} = \frac{\sum_i |A_i|}{\sum_i |G_i|}, \tag{5}$$

$$\mathit{TypeMatch} = \frac{\sum_i |\{s \in A_i : \tau_{\text{pred}}(i,s) = \tau_{\text{slot}}(s)\}|}{\sum_i |A_i|}, \tag{6}$$

where $\tau_{\text{pred}}$ and $\tau_{\text{slot}}$ are the mention and slot coarse types. *Exact20 (on hits)* is computed only on schema-hit queries with a comparable predicted/gold numeric pair,

using

$$\text{error}(i,s) = \frac{|\hat{y}_{i,s} - y_{i,s}|}{\max(1.0, |y_{i,s}|)}, \tag{7}$$

and counting a slot correct if $\text{error}(i,s) \leq 0.20$. The denominator is method-dependent (main table: 291/291/303/320 comparable pairs for TF–IDF baseline, TF–IDF ratio-aware, BGE-M3, and Oracle), so Oracle need not dominate TF–IDF on this conditional metric. Exact20 is a conditional value-grounding measure; later sections rely on this definition.

*InstantiationReady* uses per-query rates over $E(q_i, \hat{s}_i)$ (Eq. (2)): $\mathit{Coverage}_i = |A_i|/|E|$ (0 if $E = \emptyset$) and $\mathit{TypeMatch}_i = |\{s \in A_i : \tau_{\text{pred}} = \tau_{\text{slot}}\}|/|A_i|$ (0 if $A_i = \emptyset$), with

$$\mathit{InstantiationReady}_i = \mathbb{1}[\mathit{Coverage}_i \geq 0.8 \ \wedge \ \mathit{TypeMatch}_i \geq 0.8]\,. \tag{8}$$

The reported score is the mean over the 331-query denominator. The 0.8/0.8 threshold is a fixed diagnostic criterion; exact schema match is not a hard gate here (see StrictInstantiationReady in Section 4.6).

Unless noted, retrieval and readiness metrics use the full denominator; Exact20 is the sole conditional exception. We report paired bootstrap/McNemar tests and overlap sanitization diagnostics (Section 4.6), the numeric-extraction ablation (Section 4.5), and three restricted structural/solver studies (Section 4.8). Table 3 summarizes blocks. Classical retrieval and grounding are CPU-compatible; BGE-M3 used GPU float32 L2-normalized 1024-d embeddings and cosine similarity without fine-tuning (Hugging Face `datasets` [17], SCIKIT-LEARN [22]). Implementation details and reproduction commands are in the repository reproducibility map.

## 4.2 Schema Retrieval Performance

Table 4 reports Schema R@1 across query variants for the 335-way catalog (random chance $\approx 0.0030$). Within this fixed catalog—not as an open-domain claim—retrieval is already strong: BGE-M3 reaches 0.9456 on orig versus TF–IDF 0.9094 (McNemar $p = 0.0075$; 15 BGE-only vs. 3 TF–IDF-only discordant wins). Noisy results remain close to orig; short truncations drop all methods but keep them well above chance (BGE-M3 0.8157). Average ranks: BGE-M3 0.9013, TF–IDF 0.8641, BM25 0.8469, LSA 0.8328. We retain TF–IDF as the primary lexical downstream backbone and also evaluate BGE-M3 downstream to test whether residual grounding difficulty persists under stronger retrieval. Absolute levels in Table 4 set the paper's interpretive baseline: in this catalog, scalar grounding is the larger residual challenge.

## 4.3 Downstream Utility for Problem Instantiation

Table 5 reports scalar-grounding outcomes on orig queries. TF–IDF with ratio-aware extraction reaches Coverage 0.8886, TypeMatch 0.8665, and InstantiationReady 0.8006; BGE-M3 with the same grounding improves InstantiationReady to 0.8248 (Strict 0.8006). TF–IDF ratio-aware remains strongest on Exact20 (on hits) at 0.2614. Adding the schema gate (Eq. (9)) lowers TF–IDF ratio-aware readiness from 0.8006 to Strict 0.7704, so a modest number of wrong-schema queries still meet ordinary thresholds. Replacing TF–IDF with BGE-M3 or with Oracle-TG (InstantiationReady

**Table 3** Summary of the experimental blocks used in the paper.

| Block | Primary purpose | Main outputs |
|---|---|---|
| Main benchmark | Retrieval and scalar grounding on NLP4LP | Schema R@1, Coverage, TypeMatch, Exact20 (on hits), InstantiationReady |
| Robustness analyses | Sensitivity to degraded input and lexical overlap | Orig / noisy / short retrieval comparisons, overlap-based stress tests |
| Numeric-extraction ablation | Isolate effect of corrected ratio-word extraction | TypeMatch and InstantiationReady changes (baseline vs. ratio-aware extraction) |
| Significance testing | Assess whether reported gaps are statistically robust | Paired bootstrap $p$-values and confidence intervals |
| Structural validation (60 inst.) | Assess downstream structural usefulness | Structural-validity and instantiation-completeness rates |
| Extended structural validation (269 inst.) | Assess structural behavior on a larger subset with available reference code | Schema hit, structural validity, and instantiation completeness |
| Restricted solver-backed execution (20 inst.) | Real, restricted solver execution via SciPy HiGHS shim | Executable, solver-success, feasible, and objective-produced rates |

**Table 4** Schema retrieval accuracy across query variants, reported as *Schema R@1*. Each value is computed over all 331 test queries in the corresponding variant. The random baseline is the theoretical top-1 chance rate under uniform schema selection over the 335 candidate schemas, i.e., $1/335 \approx 0.0030$. Best result in each column is shown in bold.

| Baseline | Orig | Noisy | Short | Avg. |
|---|---|---|---|---|
| Random | 0.0030 | 0.0030 | 0.0030 | 0.0030 |
| LSA | 0.8459 | 0.8882 | 0.7644 | 0.8328 |
| BM25 | 0.8822 | 0.8912 | 0.7674 | 0.8469 |
| TF–IDF | 0.9094 | 0.9033 | 0.7795 | 0.8641 |
| BGE-M3 (dense) | **0.9456** | **0.9426** | **0.8157** | **0.9013** |

0.8489) confirms that retrieval helps but does not close the gap: residual difficulty is value-to-slot grounding. Exact20 ordering (BGE-M3 0.2436, Oracle 0.2505) reflects method-dependent denominators (Section 4.1), not a claim that weaker retrieval improves value accuracy. These results characterize the restricted intermediate task, not full semantic understanding or open-domain model recovery.

**Table 5** Main downstream results on the original queries, computed from the final evaluation records. *Coverage*, *TypeMatch*, *InstantiationReady*, and *StrictInstantiationReady* use the full 331-query denominator. *Exact20* (on hits) is conditional on schema-hit cases with comparable predicted and gold numeric values. Best non-oracle result in each column is shown in bold.

| Method | Coverage | TypeMatch | Exact20 | InstReady | Strict |
|---|---|---|---|---|---|
| TFIDF-TG (baseline extr.) | 0.8794 | 0.8515 | 0.2449 | 0.7764 | 0.7462 |
| TFIDF-TG (ratio-aware extr.) | 0.8886 | 0.8665 | **0.2614** | 0.8006 | 0.7704 |
| BGE-M3 + ratio-aware | **0.9154** | **0.8946** | 0.2436 | **0.8248** | **0.8006** |
| Oracle-TG | 0.9416 | 0.9230 | 0.2505 | 0.8489 | 0.8489 |

Abbreviations: TFIDF-TG = TF–IDF typed-greedy grounding; Oracle-TG = oracle schema selection with typed-greedy grounding; BGE-M3 = pretrained dense text encoder. All values are supported by the accompanying repository artifacts. *Exact20 (on hits)* uses a single, uniformly applied denominator convention across all four rows: the mean of the per-query bounded-error indicator (Eq. (7)) over exactly the schema-hit queries that have a comparable predicted/gold numeric value, excluding (not zero-filling) schema-hit queries with no comparable value. The four rows accordingly condition on 291, 291, 303, and 320 comparable queries, respectively.

## 4.4 Matched Same-Task Grounding Baselines

Table 6 compares predeclared, mechanism-diverse grounding alternatives under fixed TF–IDF retrieval, the same 331 queries and catalog, and shared InstantiationReady / Exact20 definitions (no tuning on test labels). Typed greedy and constrained assignment share the ratio-aware extractor; other rows use intrinsic enriched extractors required by their scoring definitions. Typed greedy remains best on InstantiationReady (0.8006) and StrictInstantiationReady (0.7704); all alternatives are significantly worse on InstantiationReady (paired bootstrap $B = 10{,}000$, seed 42; $p \leq 0.0006$), with McNemar likewise favoring typed greedy on Strict. Higher Exact20 for some alternatives (e.g., search-structured 0.6397) comes with lower Coverage—an abstention tradeoff, not a reason to replace the production method.

## 4.5 Error Analysis and Ablation Studies

Table 7 partitions all 331 orig queries into mutually exclusive categories (fixed order, one label each) for TF–IDF ratio-aware typed greedy: wrong schema 30 (9.1%); incomplete coverage 31 (9.4%); type mismatch 49 (14.8%); value inaccurate 214 (64.7%); fully correct 7 (2.1%). Counts are deterministic from frozen per-query records. The dominant residual is fine-grained value-to-slot correctness among type-compatible mentions—the ambiguity Exact20 (Section 4.1) targets—not coarse typing or retrieval.

Table 8 shows the ratio-word extraction ablation: Coverage 0.8794 → 0.8886, TypeMatch 0.8515 → 0.8665, InstantiationReady 0.7764 → 0.8006, Strict 0.7462 → 0.7704 (8 strict gains, 0 losses; McNemar $p = 0.008$). Retrieval is unchanged. Gains are real but modest; the remaining oracle gap reflects semantic ambiguities (totals vs. unit coefficients, bounds, percent vs. absolute values) rather than the extraction patch.

**Table 6** Matched same-task grounding baselines on the 331 original queries with TF–IDF retrieval held fixed. Only the grounding/assignment mechanism changes. Best InstantiationReady / StrictInstantiationReady values are shown in bold.

| Grounding method | Coverage | TypeMatch | Exact20 | InstReady | Strict |
|---|---|---|---|---|---|
| Typed greedy (production) | 0.8886 | 0.8665 | 0.2614 | **0.8006** | **0.7704** |
| Constrained assignment | 0.8531 | 0.8745 | 0.5772 | 0.7613 | 0.7311 |
| Max-weight matching | 0.8436 | 0.8740 | 0.5225 | 0.7432 | 0.7130 |
| Optimization-role repair | 0.8436 | 0.8724 | 0.5210 | 0.7372 | 0.7069 |
| Search-structured grounding | 0.8195 | 0.8795 | 0.6397 | 0.7039 | 0.6737 |
| Semantic-IR repair | 0.8131 | 0.8483 | 0.4949 | 0.7160 | 0.6949 |

Typed greedy and constrained assignment share the final ratio-aware extractor; the other rows use their intrinsic enriched extractors as part of their grounding definitions. *Exact20 (on hits)* denominators are 291 comparable schema-hit queries for all rows except semantic-IR repair (285), under the same exclude-noncomparable convention as Table 5.

**Table 7** Mutually exclusive residual-error decomposition for TF–IDF ratio-aware typed greedy on 331 orig queries (fixed category order; counts sum to 331).

| Category | Count | Fraction |
|---|---|---|
| Wrong schema (schema retrieval fails) | 30 | 0.091 |
| Incomplete coverage (correct schema, $\geq$ 1 slot unfilled) | 31 | 0.094 |
| Type mismatch (correct schema, full coverage, $\geq$ 1 wrong type) | 49 | 0.148 |
| Value inaccurate (correct schema, full coverage, correct types, $\geq$ 1 value outside tolerance) | 214 | 0.647 |
| Fully correct (all four criteria satisfied) | 7 | 0.021 |
| Total | 331 | 1.000 |

## 4.6 Statistical Significance and Lexical-Overlap Robustness

Table 9 summarizes paired bootstrap tests ($B$ = 10,000, seed 42) on orig queries: TF–IDF ratio-aware vs. Oracle InstantiationReady and Strict gaps are significant; the ratio-aware strict gain ($\Delta = +0.0242$; McNemar $p = 0.008$) and BGE-M3 vs. TF–IDF InstantiationReady/Strict gains are likewise significant. Direction and magnitude are readable from the table.

Table 10 shows that stripping numbers and/or stopwords does not destroy lexical retrieval; performance is preserved or improved, consistent with reliance on domain terms rather than numeric literals. The diagnostic uses a closely matched index (331 documents excluding four filler templates; no `doc_id` tokens), so TF–IDF/BM25 baseline cells differ slightly from Table 4 while LSA matches exactly (0.8459). A

**Table 8** Numeric-extraction ablation on the original queries, comparing the baseline and ratio-aware extraction configurations of the TF–IDF typed-greedy method. The ratio-aware configuration corrects multiplicative ratio-word numeric expressions. The strict-ready count (of 331) rises from 247 to 255; see Section 4.6 for the significance test.

| Metric | Baseline | Ratio-aware | Δ |
|---|---|---|---|
| Coverage | 0.8794 | 0.8886 | +0.0092 |
| TypeMatch | 0.8515 | 0.8665 | +0.0150 |
| Exact20 (on hits) | 0.2449 | 0.2614 | +0.0165 |
| InstantiationReady | 0.7764 | 0.8006 | +0.0242 |
| StrictInstantiationReady | 0.7462 | 0.7704 | +0.0242 |

**Table 9** Paired bootstrap significance tests ($B = 10{,}000$ resamples, original queries) computed from the final per-query evaluation records. Diff is method A minus method B on the stated metric.

| Comparison (metric) | Diff | 95% CI | $p$ |
|---|---|---|---|
| TFIDF-TG (ratio) vs. Oracle-TG (InstReady) | −0.0483 | [−0.0755, −0.0242] | $< 0.001$ |
| TFIDF-TG (ratio) vs. Oracle-TG (Strict) | −0.0785 | [−0.1088, −0.0514] | $< 0.001$ |
| TFIDF-TG baseline vs. ratio (Strict) | +0.0242 | [0.0091, 0.0423] | 0.0006 |
| BGE-M3 vs. TFIDF-TG (ratio) (InstReady) | +0.0242 | [0.0060, 0.0423] | 0.0053 |
| BGE-M3 vs. TFIDF-TG (ratio) (Strict) | +0.0302 | [0.0060, 0.0544] | 0.0142 |

The baseline versus ratio-aware extraction comparison (8 strict gains, 0 losses) is also significant under an exact McNemar test ($p = 0.008$). CI = confidence interval. "ratio" abbreviates the ratio-aware extraction configuration. All five rows are reproducible from a single committed, deterministic script (paired percentile bootstrap, $B = 10{,}000$, seed 42, two-sided $p = 2\min(P(\Delta \le 0), P(\Delta > 0))$ over the bootstrap resamples) applied to the frozen per-query records; see Code availability.

Jaccard-overlap stratification places 327/331 queries in a high-overlap bucket (TF–IDF R@1 0.9174); the medium-overlap bucket has only 4 queries and is reported only as a qualitative caveat.

To check whether wrong schemas can inflate ordinary InstantiationReady via smaller eligible-slot sets, a repository diagnostic of selective schema reranking raised ordinary readiness from 257/331 to 265/331, but 6 of 8 newly ready queries used an incorrect schema. We therefore define

$$\mathit{StrictInstantiationReady}_i = \mathbb{1}[\mathit{SchemaHit}_i \wedge \mathit{Coverage}_i \ge 0.8 \wedge \mathit{TypeMatch}_i \ge 0.8], \tag{9}$$

as a robustness check (Table 11). The gate removes 10 TF–IDF queries in both extraction configurations and widens the TF–IDF–Oracle InstantiationReady gap from 0.0483 to 0.0785 ($p < 0.001$), without changing the qualitative conclusion.

**Table 10** Retrieval accuracy (*Schema R@1*, original queries) after removing numeric tokens and/or stopwords, using the diagnostic sanitization configuration with 256-dimensional LSA. The TF–IDF and BM25 baseline rows differ slightly from the canonical Table 4 evaluation because of the catalog and indexed-document configuration differences described in the text; the LSA baseline reproduces the Table 4 value exactly.

| Sanitization | TF–IDF | BM25 | LSA |
|---|---|---|---|
| None (baseline) | 0.9063 | 0.8852 | 0.8459 |
| Numbers removed | 0.9063 | 0.8973 | 0.8459 |
| Stopwords removed | 0.9124 | 0.9063 | 0.9184 |
| Numbers & stopwords removed | 0.9124 | 0.9063 | 0.9184 |

**Table 11** Sensitivity check: canonical *InstantiationReady* versus the stricter *StrictInstantiationReady* (Eq. (9)), original queries, computed from the final per-query evaluation records. $\Delta$ is standard minus strict; $n_{\text{differ}}$ counts queries whose readiness label changes.

| Method | InstReady | StrictInstReady | $\Delta$ | $n_{\text{differ}}$ |
|---|---|---|---|---|
| TFIDF-TG (baseline) | 0.7764 | 0.7462 | 0.0302 | 10 |
| TFIDF-TG (ratio-aware) | 0.8006 | 0.7704 | 0.0302 | 10 |
| Oracle-TG | 0.8489 | 0.8489 | 0.0000 | 0 |

## 4.7 Computational Complexity and Runtime

TF–IDF retrieval is $O(\text{nnz}(q) + M)$ per query ($M = 335$); rule extraction is linear in query length; typed greedy is $O(|\mathcal{P}| \cdot |\mathcal{T}|)$ with small slot/mention counts. On the frozen TF–IDF + ratio-aware typed-greedy run (331 queries, one CPU core, `PYTHONHASHSEED=0`), wall time is 1.09 s (3.29 ms/query) with peak RSS $\approx$ 198 MB. This excludes BGE-M3 embedding load/compute and remote LLM baselines; it substantiates that the deterministic backbone is lightweight and CPU-compatible, not a cross-method efficiency benchmark.

## 4.8 Structural and Solver-Backed Validation

Core metrics are solver-free. We complement them with three restricted studies on subsets with available `optimus_code`: structural analyses on 60 and 269 instances (static; no Gurobi required) and solver-backed execution on 20 HiGHS-compatible instances (SciPy HiGHS shim). All three are restricted by construction and do not claim benchmark-wide executability.

*Structural validity*: well-formed LP/MILP with objective and $\geq 1$ variable (static). *Instantiation completeness*: every gold-required scalar slot receives a type-consistent assignment. *Executable / solver success / feasible / objective produced*: runtime stages as named.

**Table 12** Structural subset (60 instances): schema-hit, structural-validity, and instantiation-completeness rates.

| Baseline | Schema Hit | Structural Valid | Inst. Complete |
|---|---|---|---|
| TF–IDF | 0.9333 | 0.7500 | 0.7500 |
| BM25 | 0.9000 | 0.7333 | 0.7333 |
| Oracle | 1.0000 | 0.7667 | 0.7833 |

**Table 13** Restricted solver-backed subset (20 instances, SciPy HiGHS MILP compatibility shim; Gurobi not required).

| Baseline | Executable | Solver Success | Feasible | Objective Produced |
|---|---|---|---|---|
| TF–IDF | 0.95 | 0.80 | 0.80 | 0.80 |
| Oracle | 0.95 | 0.75 | 0.75 | 0.75 |

***Structural subset (60)***

Table 12: TF–IDF structural-valid and instantiation-complete both 0.75; Oracle 0.7667 / 0.7833. Oracle gains remain modest.

***Extended structural subset (269)***

With available reference code, TF–IDF / BM25 / Oracle schema-hit rates are 0.9368 / 0.9257 / 1.0000; structural-valid 0.8141 / 0.8104 / 0.8253; instantiation-complete 0.6654 / 0.6580 / 0.6840. Solver execution was not evaluated here (reference path required unavailable Gurobi support); executable claims are reserved for the HiGHS subset.

***Restricted solver-backed subset (20)***

The first 20 test-split instances whose gold code lacks unsupported commercial constructs under a fixed, outcome-independent filter (identical IDs for TF–IDF and Oracle; chosen before evaluating either). Table 13: TF–IDF 0.95 executable and 0.80 on success/feasible/objective; Oracle 0.95 / 0.75 / 0.75 / 0.75. Non-executions are shim `IndexErrors` (2 of 40 baseline–instance evaluations). The 0.80 vs. 0.75 gap is inconclusive on $n = 20$ compatibility-filtered data—a pragmatic demonstration that execution is possible on compatible cases, not benchmark-wide readiness.

Across subsets, Oracle does not consistently dominate TF–IDF, matching the main-benchmark pattern that schema selection yields only modest gains while downstream limits remain. Extended-subset structural gaps (missing slots, type mismatch, incomplete instantiation: 69, 82, and 267 counted instances across baselines) align with Section 4.5.

**Table 14** External-baseline provenance on the common-18 subset. Provenance: NATIVE / INDEPENDENT / ADAPTED-OFFICIAL / GENERIC-ZERO-SHOT. Ours is listed for context only; native metrics are not comparable to Table 15.

| System | Provenance | Output representation | Solver environment / status | Cases |
|---|---|---|---|---|
| Ours (this paper) | NATIVE | Scalar slot assignments on retrieved schema | No solver call in core benchmark | 18 |
| PaMOP [21] | INDEPENDENT | AMPL formulation + solver code | Local AMPL + HiGHS; 13/18 executed | 18 |
| ORLM [13] | ADAPTED-OFFICIAL | LP formulation + solver code | Solver dependency (`coptpy`) unavailable; execution blocked | 18 |
| OptMATH [20] | ADAPTED-OFFICIAL | LP formulation + solver code | Gurobi (`gurobipy`); 15/18 executed | 18 |
| Generic LLM (GPT-5.4-2026-03-05) | GENERIC-ZERO-SHOT | Gurobi solver code, zero-shot | Gurobi (`gurobipy`); 16/18 executed | 18 |
| DeepOR [31] | — | Formulation (paper-described) | No official code or released checkpoint located; no evaluable artifact exists | 0 |
| OR-R1 [11] | — | Formulation + solver code | Official code public; no checkpoint released; execution not possible | 0 |

## 4.9 External Baseline Comparison on a Shared Subset

The proposed method performs fixed-catalog scalar grounding and does not emit complete formulations or solver code; its native metrics are therefore not numerically comparable to end-to-end LLM systems. Tables 14–15 provide contextual common-18 evidence under a shared harness (full protocol in the repository). Fairness notes: (i) different tasks (Ours excluded from end-to-end cells); (ii) PaMOP is an independent Azure OpenAI reconstruction (`gpt-5.4-2026-03-05`, 2026-08-15, temperature 0.2), with objective agreement within 5% relative tolerance on evaluable successful runs; (iii) ORLM execution is N/A (`coptpy` unavailable); DeepOR has no located code/checkpoint and OR-R1 has public code but no released checkpoint—neither yields an evaluable artifact; (iv) the generic LLM is zero-shot under the harness (temperature 0.0), not an optimization-trained method, and must not be ranked against OptMATH's official checkpoint (temperature 0.8); (v) $n = 18$ is a feasibility probe, not a leaderboard.

**Table 15** Shared outcomes on the 18-instance common subset. N/A = not evaluated (never a silent zero). Parse / Executable / Feasible / Objective agreement as defined in the harness; denominators vary by stage reached.

| System | Parse | Executable | Feasible | Objective agreement |
|---|---|---|---|---|
| Ours (this paper) | N/A | N/A | N/A | N/A |
| PaMOP [21] | 0.72 | 0.72 | 1.00 | 0.73 (8/11) |
| ORLM [13] | 1.00 | N/A | N/A | N/A |
| OptMATH [20] | 1.00 | 0.83 | 1.00 | 0.40 (6/15) |
| Generic LLM (GPT-5.4-2026-03-05) | 1.00 | 0.89 | 1.00 | 0.63 (10/16) |

### 4.10 External Component-Level Robustness Validation

As a component-level check, we evaluated numeric extraction on a fixed 1,000-instance OptMATH-Train sample [20] (no fixed schema catalog, so extraction only). Raw AST code-value overlap recall is 0.7501; automatic text-support recall 0.8118 (complete-instance rate 0.5910). A secondary rule-based classifier on 150 instances (seed 20260815) finds that 7.28% of the AST denominator are implementation/loop/solver constants. Audit-calibrated text-supported recall is 0.8070 (95% CI $[0.7842, 0.8283]$, $n = 28{,}799$ auto-classified text-supported values). Two independently coded classifiers agree on 98.04% of 4,801 literals in the 150-instance sample—automated cross-validation, not human gold labels. This is limited evidence that extraction transfers beyond NLP4LP; it does not establish full-framework generalization.

## 5 Conclusions and Future Research

We studied fixed-catalog schema retrieval paired with deterministic scalar grounding as an intermediate natural-language-to-optimization capability. On NLP4LP, lexical and dense retrieval are already strong; BGE-M3 improves InstantiationReady, yet the oracle gain remains modest, and residual-error analysis attributes most remaining failures to fine-grained value-to-slot correctness. Matched same-task grounding alternatives do not displace typed greedy on InstantiationReady/Strict. Restricted structural and solver-backed subsets suggest usefulness on compatible cases without establishing benchmark-wide solver readiness. Limitations are consolidated in Section 5.1.

### 5.1 Limitations and Threats to Validity

*Fixed-catalog retrieval and task scope.* Strong retrieval numbers characterize the 335-way NLP4LP catalog (Section 3.2) and must not be read as solving open-domain schema retrieval.

*Dataset dependence and lexical overlap.* All main quantitative results use a single benchmark. Overlap sanitization (Section 4.6) argues against pure numeric-literal matching, but the medium-overlap bucket has only 4 queries; cross-style/domain/language generalization is untested.

*Gated data access.* NLP4LP requires approved Hugging Face access. Committed artifacts support inspection without live queries; full end-to-end reruns need the gated data.

*Scalar-only instantiation.* Coverage / TypeMatch / InstantiationReady characterize scalar recovery only (Section 3.4).

*Heuristic type and role inference.* Coarse typing and optional role cues are rule-based without formal guarantees; Table 7 shows value-to-slot ambiguity remains dominant.

*Restricted solver-backed validation.* Subsets of size 60 / 269 / 20 are availability- or compatibility-filtered, not random; HiGHS only; the 20-case study cannot support benchmark-wide readiness claims (Section 4.8).

*Limited end-to-end LLM comparison.* Related Work and the common-18 assessment contextualize OptiMUS, OptLLM, ORLM, OptMATH, OptimAI, AlphaOPT, ORThought, and others, but do not provide full-benchmark InstantiationReady head-to-heads; DeepOR/OR-R1 lack evaluable artifacts (Section 4.9).

*No external full-framework validation or deployment.* OptMATH validates extraction only (Section 4.10); no user study or production deployment was conducted.

### 5.2 Future Research Directions

Natural next steps include stronger semantic role-sensitive grounding, structured (non-scalar) parameters, hybrid retrieval-plus-learned/LLM grounding, and broader solver-backed coverage. Intermediate schema retrieval and schema-conditioned instantiation remain scientifically useful even while full automation stays difficult.

## Declarations

**Funding.** This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The external large-language-model baseline experiments used Microsoft Azure OpenAI services; associated usage was supported by the standard Azure for Students educational credit available to the author.

**Competing interests.** The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

**Ethics approval and consent to participate.** Not applicable; this study did not involve human participants, human data, or animals.

**Author contributions.** Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. This single-author paper uses the standard CRediT taxonomy to describe the contributions of the sole author.

**Data availability.** The benchmark used in this paper, NLP4LP (`udell-lab/NLP4LP` on Hugging Face), is a gated third-party dataset. The author does not redistribute this dataset; eligible researchers must obtain it directly from its original Hugging Face source under its specific access conditions. The derived artifacts supporting the results reported in this paper, together with the corresponding tables, figures, and

analysis outputs, are publicly available in the accompanying GitHub repository at https://github.com/SoroushVahidi/combinatorial-opt-agent.

**Code availability.** The retrieval, grounding, evaluation, and analysis code supporting this paper is available at https://github.com/SoroushVahidi/combinatorial-opt-agent. Reproducing a full NLP4LP rerun requires the gated dataset access described above.

**Declaration of generative AI and AI-assisted technologies in the writing process.** During the preparation of this work the author used ChatGPT and Gemini to improve readability and language. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

**Declaration of AI assistance in software development.** During the research and software-development process, the author used Claude, Cursor, and GitHub Copilot for coding assistance. The author reviewed, verified, tested, and edited the resulting code and takes full responsibility for the implementation and reported results. Note that the scalar grounding procedure evaluated in this paper is deterministic at inference time and does not itself rely on generative LLM inference; schema retrieval is evaluated using both deterministic lexical methods and the pretrained BGE-M3 dense encoder (with the exception of the external baselines evaluated separately in Section 4.9).

# References


[1] Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. The VLDB Journal 28(5):793–819. https://doi.org/10.1007/s00778-019-00567-8

[2] AhmadiTeshnizi A, Gao W, Udell M (2023) NLP4LP: A dataset for natural language to linear programming and mixed-integer linear programming. Hugging Face dataset repository, URL https://huggingface.co/datasets/udell-lab/NLP4LP, available at the udell-lab/NLP4LP dataset page

[3] AhmadiTeshnizi A, Gao W, Udell M (2024) OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In: Proceedings of the 41st International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 235. PMLR, pp 577–596, URL https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html

[4] Ahmed T, Choudhury S (2025) OPT2CODE: A retrieval-augmented framework for solving linear programming problems. Natural Language Processing Journal 13:100185. https://doi.org/10.1016/j.nlp.2025.100185, URL https://www.sciencedirect.com/science/article/pii/S2949719125000615

[5] Astorga N, Liu T, Xiao Y, et al (2025) Autoformulation of mathematical optimization models using LLMs. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning

Research, vol 267. PMLR, pp 1864–1886, URL https://proceedings.mlr.press/v267/astorga25a.html

[6] Atzeni P, Baldazzi T, Bellomarini L, et al (2026) Semantic-aware query answering with large language models. Data & Knowledge Engineering 161:102494. https://doi.org/10.1016/j.datak.2025.102494

[7] Biemans B, Troubil P, Grau I, et al (2026) Explainable optimization: Leveraging large language models for user-friendly explanations. In: Guidotti R, Schmid U, Longo L (eds) Explainable Artificial Intelligence, Communications in Computer and Information Science, vol 2578. Springer, Cham, p 44–67, https://doi.org/10.1007/978-3-032-08327-2_3, URL https://link.springer.com/chapter/10.1007/978-3-032-08327-2_3, xAI 2025, first online 12 October 2025

[8] Chen J, Xiao S, Zhang P, et al (2024) M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of the Association for Computational Linguistics: ACL 2024. Association for Computational Linguistics, pp 2318–2335, https://doi.org/10.18653/v1/2024.findings-acl.137

[9] Chen Y, Xia J, Shao S, et al (2025) Solver-informed RL: Grounding large language models for authentic optimization modeling. In: Advances in Neural Information Processing Systems (NeurIPS)

[10] Deerwester S, Dumais ST, Furnas GW, et al (1990) Indexing by latent semantic analysis. Journal of the American Society for Information Science 41(6):391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9

[11] Ding Z, Tan Z, Zhang J, et al (2026) OR-R1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 228–236, https://doi.org/10.1609/aaai.v40i1.36983

[12] Gao Z, Ge A, Berenbeim A, et al (2026) Models can model, but can't bind: Structured grounding in text-to-optimization. In: Conference on Language Modeling (COLM)

[13] Huang C, Tang Z, Hu S, et al (2025) ORLM: A customizable framework in training large models for automated optimization modeling. Operations Research 73(6):2986–3009. https://doi.org/10.1287/opre.2024.1233

[14] Jiang C, Shu X, Qian H, et al (2025) LLMOPT: Learning to define and solve general optimization problems from scratch. In: Proceedings of the Thirteenth International Conference on Learning Representations, URL https://openreview.net/forum?id=9OMvtboTJg

[15] Kadioglu S, Dakle PP, Uppuluri K, et al (2024) Ner4Opt: Named entity recognition for optimization modelling from natural language. Constraints 29(3–4):261–299. https://doi.org/10.1007/s10601-024-09376-5

[16] Kong M, Qu A, Guo X, et al (2025) AlphaOPT: Formulating optimization programs with self-improving LLM experience library. arXiv preprint arXiv:251018428 URL https://arxiv.org/abs/2510.18428, arXiv:2510.18428 [cs.AI]

[17] Lhoest Q, Villanova del Moral A, Jernite Y, et al (2021) Datasets: A community library for natural language processing. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp 175–184, https://doi.org/10.18653/v1/2021.emnlp-demo.21, URL https://aclanthology.org/2021.emnlp-demo.21/

[18] Li Y, Wang H, Xue B, et al (2026) Solver-independent automated problem formulation via LLMs for high-cost simulation-driven design. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.102

[19] Li Z, Lu H, Wei T, et al (2026) Constructing industrial-scale optimization modeling benchmark. In: Proceedings of the 43rd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 306. PMLR

[20] Lu H, Xie Z, Wu Y, et al (2025) OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, vol 267. PMLR, pp 40769–40802, URL https://proceedings.mlr.press/v267/lu25o.html

[21] Pan X, Fang J, Wu F, et al (2025) Guiding large language models in modeling optimization problems via question partitioning. In: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25), pp 2657–2665, https://doi.org/10.24963/ijcai.2025/296, URL https://www.ijcai.org/proceedings/2025/296

[22] Pedregosa F, Varoquaux G, Gramfort A, et al (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12:2825–2830

[23] Powell C, Riccardi A (2025) Generating textual explanations for scheduling systems leveraging the reasoning capabilities of large language models. Journal of Intelligent Information Systems 63(4):1287–1337. https://doi.org/10.1007/s10844-025-00940-w, URL https://doi.org/10.1007/s10844-025-00940-w

[24] Ramamonjison R, Li H, Yu TTL, et al (2022) Augmenting operations research with auto-formulation of optimization models from problem descriptions. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language

Processing: Industry Track. Association for Computational Linguistics, pp 29–62, https://doi.org/10.18653/v1/2022.emnlp-industry.4, URL https://aclanthology.org/2022.emnlp-industry.4/

[25] Ramamonjison R, Yu T, Li R, et al (2023) NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In: Proceedings of the NeurIPS 2022 Competitions Track, Proceedings of Machine Learning Research, vol 220. PMLR, pp 189–203, URL https://proceedings.mlr.press/v220/ramamonjison23a.html

[26] Rizzi S, Francia M, Gallinucci E, et al (2025) Conceptual design of multidimensional cubes with LLMs: An investigation. Data & Knowledge Engineering 159:102452. https://doi.org/10.1016/j.datak.2025.102452

[27] Robertson S, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4):333–389. https://doi.org/10.1561/1500000019

[28] Sheetrit E, Brief M, Mishaeli M, et al (2024) ReMatch: Retrieval enhanced schema matching with LLMs. arXiv preprint arXiv:240301567 https://doi.org/10.48550/arXiv.2403.01567, URL https://arxiv.org/abs/2403.01567

[29] Thind R, Sun Y, Liang L, et al (2025) OptimAI: Optimization from natural language using LLM-powered AI agents. arXiv preprint arXiv:250416918 URL https://arxiv.org/abs/2504.16918, arXiv:2504.16918 [cs.CL]

[30] Xiao Z, Zhang D, Wu Y, et al (2024) Chain-of-experts: When LLMs meet complex operations research problems. In: Proceedings of the Twelfth International Conference on Learning Representations, URL https://openreview.net/forum?id=HobyL1B9CZ

[31] Xiao Z, Wang YJ, Han X, et al (2026) DeepOR: A deep reasoning foundation model for optimization modeling. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 34052–34060, https://doi.org/10.1609/aaai.v40i40.40699

[32] Xie W, Chen Y, Hu YX, et al (2026) Escaping the sisyphus dilemma: Experience replay for robust text-to-optimization modeling. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.690

[33] Yang B, Zhou Q, Li J, et al (2026) ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. Artificial Intelligence for Transportation 6:100059. https://doi.org/10.1016/j.ait.2026.100059, URL https://doi.org/10.1016/j.ait.2026.100059

[34] Yu L, Zhao X, Li D, et al (2026) Uncertainty-aware test-time search for optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.1975

[35] Yu T, Zhang R, Yang K, et al (2018) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, pp 3911–3921, https://doi.org/10.18653/v1/D18-1425, URL https://aclanthology.org/D18-1425/

[36] Zhai H, Lawless C, Vitercik E, et al (2025) EquivaMap: Leveraging LLMs for automatic equivalence checking of optimization formulations. In: Proceedings of the 42nd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 267. PMLR, pp 74288–74305

[37] Zhang J, Wang W, Guo S, et al (2024) Solving general natural-language-description optimization problems with large language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track). Association for Computational Linguistics, pp 483–490, https://doi.org/10.18653/v1/2024.naacl-industry.42, URL https://aclanthology.org/2024.naacl-industry.42/

[38] Zhang X, Wan Y, Zhang B, et al (2026) Dual-cluster memory agent: Resolving multi-paradigm ambiguity in optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.266